RDF 1.2 Interoperability

W3C Editor's Draft 15 December 2025

▼ **More details about this document**

This version:

https://w3c.github.io/rdf-interop/spec/

Latest published version:

https://www.w3.org/TR/rdf12-interop/

Latest editor's draft:

https://w3c.github.io/rdf-interop/spec/

History:

https://www.w3.org/standards/history/rdf12-interop/

Commit history

Editor:

Pierre-Antoine Champin

Feedback:

GitHub w3c/rdf-interop (pull requests, new issue, open issues)

Copyright © 2025 World Wide Web Consortium. W3C® liability, trademark and permissive document license rules apply.

Abstract

The goal of this specification is to provide guidance and good practices to achieve interoperability across different versions or profiles of RDF.

Status of This Document

This section describes the status of this document at the time of its publication. A list of current W3C publications and the latest revision of this technical report can be found in the W3C standards and drafts index.

This document was published by the RDF & SPARQL Working Group as an Editor's Draft.

Publication as an Editor's Draft does not imply endorsement by W3C and its Members.

This is a draft document and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to cite this document as other than a work in progress.

This document was produced by a group operating under the W3C Patent Policy. W3C maintains a public list of any patent disclosures made in connection with the deliverables of the group; that page also includes instructions for disclosing a patent. An individual who has actual knowledge of a patent that the individual believes contains Essential Claim(s) must disclose the information in accordance with section 6 of the W3C Patent Policy.

This document is governed by the 18 August 2025 W3C Process Document.

§ Set of Documents

This document is one of eleven RDF 1.2 and twelve SPARQL 1.2 documents produced by the RDF & SPARQL Working Group.

▶ **List of documents**

Table of Contents

Abstract

Status of This Document
Set of Documents

1. Introduction

2. Notation and Terminology

3. Interoperability between RDF Basic and RDF Full
3.1 From Full to Basic
3.1.1 Example

3.2 From Basic to Full
3.3 Limitations
3.4 Algorithms
3.4.1 The `basic-encode` algorithm
3.4.2 The `basic-encode-triple-term` algorithm
3.4.3 The `basic-decode` algorithm

A. Acknowledgments

B. References
B.1 Informative references

§ 1. Introduction

RDF 1.2 [*RDF12-CONCEPTS*] defines two classes of conformance, namely *RDF 1.2 Full* and *RDF 1.2 Basic*, which can also be understood as two variants of the RDF 1.2 syntax. More specifically, RDF 1.2 Basic is the subset of RDF 1.2 where no triple term is used. This raises a challenge when Basic and Full implementations need to interoperate.

This specification aims to provide guidance to solve these interoperability challenges. Note that the methods and tools provided here are not normative, as they are not meant to be the only and authoritative way to achieve interoperability across heterogeneous implementations. As pointed out in the Conformance section of RDF 1.2 Concepts, different strategies are acceptable, depending on the context.

§ 2. Notation and Terminology

This document uses the following terminology for describing RDF graph syntax, all as defined in the companion RDF Concepts specification [*RDF12-CONCEPTS*]: ***appear***, ***asserted triple***, ***basic***, ***blank node***, ***default graph***, ***equivalent***, ***full***, ***named graph***, ***object***, ***predicate***, ***RDF dataset***, ***RDF graph***, ***reifier***, ***subject***, ***triple***, ***triple term***.

§ 3. Interoperability between RDF Basic and RDF Full

This section provides transformations between Full RDF graphs (respectively, RDF datasets) and Basic RDF graphs (respectively, RDF datasets), to provide some level of interoperability between the different classes of Conformance.

> ISSUE 1
>
> Should we go even further and aim to provide interoperability between *RDF 1.1* and RDF 1.2 Full?

These transformation are designed to be:

Information preserving
: It must be possible to reconstruct the input graph (respectively, dataset) from the output graph (respectively, dataset). Note, however, that these transformations are not designed to preserve semantics: the output graph is not semantically equivalent to the input graph, at least not in the entailment regimes defined in [*RDF12-SEMANTICS*].

Idempotent
: Applying a transformation several times to a graph (respectively, dataset) should have the same effect as applying it once. Moreover, basic encoding a graph (respectively, dataset) that is already complying with RDF Basic (i.e., containing no triple term) must result in the same graph (respectively, dataset).

Universal
: It should be possible to transform any Full graph (respectively, dataset) to a Basic graph (respectively, dataset) using this method. There is actually a minor caveat to this property.

§ 3.1 From Full to Basic

Encoding an RDF graph to ensure that it is consumable by an RDF Basic implementation is called ***basic encoding*** it. Basic encoding consists of repeating the following steps until no triple term appears in the graph, and the graph is therefore compliant with RDF Basic: picking a

triple term *tt* that appears in the graph; minting a fresh blank node *b* (i.e., a blank node not yet in use in the graph); replacing all occurrences of *tt* appearing in the graph with *b*; and then adding the following triples to the graph (where *s*, *p*, and *o* are respectively the subject, predicate and object of *tt*):

- (*b*, `rdf:type`, `rdf:PropositionForm`)
- (*b*, `rdf:propositionFormSubject`, *s*)
- (*b*, `rdf:propositionFormPredicate`, *p*)
- (*b*, `rdf:propositionFormObject`, *o*)

Note that this transformation is *information preserving* only when the input graph either has no triple term appearing in it, or contains no asserted triple (*b*, `rdf:type`, `rdf:PropositionForm`) where *b* is a blank node. Implementations encountering this situation ought to report an error. This limitation is discussed in Section 3.3 Limitations.

> NOTE
>
> The blank nodes generated to replace triple terms should not be confused with the reifiers that are typically associated with these triple terms.

Basic encoding an RDF dataset consists of basic encoding its default graph and each of its named graph. In this case, the fresh blank node assigned to each triple term must not be used in any graph of the dataset.

A detailed algorithm of the transformation is found in Section 3.4.1 The `basic-encode` algorithm.

§ 3.1.1 Example

The examples in this section are expressed in the Turtle concrete syntax [*RDF12-TURTLE*].

EXAMPLE 1: An input Full RDF graph

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex: <http://example.org/>

<< ex:s ex:p ex:o >> ex:q "some value".
```

EXAMPLE 2: The same graph as above, with reifiers made explicit

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex: <http://example.org/>

_:r1 rdf:reifies <<( ex:s ex:p ex:o )>>.
_:r1 ex:q "some value".
```

EXAMPLE 3: The result of basic encoding the dataset above

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX ex: <http://example.org/>

_:r1 rdf:reifies _:gen1.
_:r1 ex:q "some value".

_:gen1 a rdf:PropositionForm;
  rdf:propositionFormSubject ex:s;
  rdf:propositionFormPredicate ex:p;
  rdf:propositionFormObject ex:o.
```

§ 3.2 From Basic to Full

Reverting a basic encoded graph to its original form consists of locating each asserted triple (*b*, `rdf:type`, `rdf:PropositionForm`) that has a blank node *b* as its subject, along with the three associated asserted triples that have the same blank node *b* as their subjects, i.e., (*b*, `rdf:propositionFormSubject`, *s*), (*b*, `rdf:propositionFormPredicate`, *p*), and (*b*,

`rdf:propositionFormObject`, *o*); removing these four triples from the graph; and replacing all remaining occurrences of *b* appearing in the graph with the triple term (*s*, *p*, *o*).

An implementation ought to report an error if, for a given *b*, it can not unambiguously determine *s*, *p*, or *o* (i.e., if one of the properties of *b* — `rdf:propositionFormSubject`, `rdf:propositionFormPredicate`, or `rdf:propositionFormObject` — is missing or duplicated). An implementation also ought to report an error if the input graph contains at the same time a triple term and an asserted triple (*b*, `rdf:type`, `rdf:PropositionForm`) where *b* is the same blank node. Note that none of these situations can occur if the input graph was produced by the basic encoding transformation.

To revert a basic encoded RDF dataset to its original form, the transformation above is applied to its default graph and to each of its named graphs.

Note that this transformation has no effect on any RDF graph or RDF dataset that does not use the `rdf:PropositionForm` type, including Full graphs or datasets containing triple terms. This makes this transformation *idempotent* as intended.

§ 3.3 Limitations

The two transformations above explicitly do not support graphs or datasets containing at the same time a triple term and an asserted triple (*b*, `rdf:type`, `rdf:PropositionForm`) where *b* is a blank node. This means that the basic encoding transformation is not *strictly* universal.

This limitation should not be an issue in practice. The `rdf:PropositionForm` type is unlikely to be in used in any published graph or dataset, as it was not defined prior to this specification. For this reason, using it would actually have been bad practice. For future graphs and datasets, this type should be considered to be reserved for use within the basic encoding transformation, and not used otherwise.

NOTE

This is one reason why this transformation introduces new vocabulary terms (`rdf:PropositionForm`, `rdf:propositionFormSubject`, `rdf:propositionFormPredicate`, `rdf:propositionFormObject`), rather than repurposing the existing reification vocabulary (`rdf:Statement`, `rdf:subject`, `rdf:predicate`, `rdf:object`). Unlike `rdf:PropositionForm`, `rdf:Statement` is known to be found in widely used datasets (e.g., Uniprot), so reserving its use for the basic encoding transformation was not an option.

Another consequence of this restriction is that implementers will need to be aware and careful when merging graphs in an application that basic encoded graphs or datasets. The concern is that merging a Full RDF graph containing at least one triple term with a basic encoded RDF graph (which might contain blank node instances of `rdf:PropositionForm`) could result in a "hybrid" graph that cannot be transformed to a consistent Full nor Basic RDF graph. Therefore, such applications should basic encode every graph prior to merging them. Conversely, applications supporting RDF Full should make sure to apply the reverse transformation to any graph that is known or likely to have been basic encoded, to avoid creating such "hybrid" graphs. Since these transformations are designed to be *idempotent*, there is no harm in applying them more than necessary.

§ 3.4 Algorithms

§ 3.4.1 The `basic-encode` algorithm

The algorithm expects one input variable G_i which is an RDF graph. It returns a Basic RDF graph.

1. Let G_o be an empty RDF graph.
2. Let M be an empty map from triple terms to blank nodes.
3. Let *inputKind* be `null`.
4. For each triple (s, p, o) in G_i:

1. If *s* is a blank node, *p* is `rdf:type` and *o* is `rdf:PropositionForm`, then:
    1. If *inputKind* is `"full"` then exit with an error.
    2. Otherwise, set *inputKind* to `"basic"`.
2. If *o* is a triple term, then:
    1. If *inputKind* is `"basic"` then exit with an error.
    2. Otherwise, set *inputKind* to `"full"`.
    3. Let *b*, *M*' and *G*' be the result of invoking `basic-encode-triple-term` passing *o* as *t* and *M* as *Mi*.
    4. Merge *M*' into *M*.
    5. Merge *G*' into G_o.
    6. Set *o* to *b*.
3. Add the triple (*s*, *p*, *o*) to G_o.

5. Return G_o.

§ 3.4.2 The `basic-encode-triple-term` algorithm

This algorithm is responsible for incrementally populating the mapping *M* and the graph *G* used internally by the `basic-encode` algorithm. It receives a triple term as input and processes it recursively (in case its object is itself a triple term). It returns, among other things, the blank node minted to replace the triple term in the transformed Basic RDF graph.

This algorithm expects two input variables: a triple term *t*, and a map M_i from triple terms to blank nodes. It returns a blank node *b*, a map M_o from triple terms to blank nodes, and a Basic RDF graph *G*.

1. Let M_o be an empty map.
2. Let *G* be an empty RDF graph.
3. Let *b* be the blank node associated with *t* in M_i, if any.
4. Otherwise:
    1. Let *s*, *p* and *o* be the subject, predicate and object of *t*, respectively.

2. If o is a triple term, then:
    1. Let b', M' and G' be the result of invoking `basic-encode-triple-term` passing o as t and M_i.
    2. Set o to b'.
    3. Merge M' into M_o.
    4. Merge G' into G.
3. Let b be a fresh blank node.
4. Add the association (t, b) to M_o.
5. Add the triples (b, `rdf:type`, `rdf:PropositionForm`), (b, `rdf:propositionFormSubject`, s), (b, `rdf:propositionFormPredicate`, p), and (b, `rdf:propositionFormObject`, o) in G.

5. Return b, M_o and G.

3.4.3 The `basic-decode` algorithm

> ISSUE 2
>
> Write this algorithm

A. Acknowledgments

In addition to the editors, the following people have contributed to this specification: Ted Thibodeau Jr

Members of the RDF & SPARQL Working Group Group included Vladimir Alexiev, James Anderson, Amin Anjomshoaa, Julián Arenas-Guerrero, Dörthe Arndt, Bilal Ben Mahria, Erich Bremer, Dan Brickley, Kurt Cagle, Sarven Capadisli, Rémi Ceres, Pierre-Antoine Champin, David Chaves-Fraga, Souripriya Das, Daniil Dobriy, Enrico Franconi, Jeffrey Phillips Freeman, Fabien Gandon, Benjamin Goering, Damien Graux, Adrian Gschwend, Olaf Hartig, Timothée Haudebourg, Ian Horrocks, Gregg Kellogg, Mark Kim, Jose Emilio Labra Gayo, Ora

Lassila, Richard Lea, Niklas Lindström, Pasquale Lisena, Thomas Lörtsch, Matthew Nguyen, Peter Patel-Schneider, Thomas Pellissier Tanon, Dave Raggett, Jean-Yves ROSSI, Felix Sasaki, Andy Seaborne, Alan Snyder, Stuart Sutton, Ruben Taelman, Ted Thibodeau Jr, Dominik Tomaszuk, Raphaël Troncy, William Van Woensel, Gregory Williams, Jesse Wright, Achille Zappa, and Antoine Zimmermann.

B. References

B.1 Informative references

[RDF12-CONCEPTS]

RDF 1.2 Concepts and Abstract Data Model. Gregg Kellogg; Olaf Hartig; Pierre-Antoine Champin; Andy Seaborne. W3C. 28 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-concepts/

[RDF12-N-QUADS]

RDF 1.2 N-Quads. Gregg Kellogg; Dominik Tomaszuk. W3C. 22 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-quads/

[RDF12-N-TRIPLES]

RDF 1.2 N-Triples. Gregg Kellogg; Dominik Tomaszuk. W3C. 20 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-n-triples/

[RDF12-NEW]

What's New in RDF 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/rdf-new/spec/

[RDF12-PRIMER]

RDF 1.2 Primer. Niklas Lindström; Pierre-Antoine Champin. W3C. 3 April 2025. DNOTE. URL: https://www.w3.org/TR/rdf12-primer/

[RDF12-SCHEMA]

RDF 1.2 Schema. Dominik Tomaszuk. W3C. 26 December 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-schema/

[RDF12-SEMANTICS]

RDF 1.2 Semantics. Peter Patel-Schneider; Dörthe Arndt; Enrico Franconi. W3C. 1 February 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-semantics/

[RDF12-TRIG]

RDF 1.2 TriG. Gregg Kellogg; Dominik Tomaszuk. W3C. 13 November 2025. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-trig/

[RDF12-TURTLE]

RDF 1.2 Turtle. Gregg Kellogg; Dominik Tomaszuk. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-turtle/

[RDF12-XML]

RDF 1.2 XML Syntax. Gregg Kellogg. W3C. 2 February 2026. W3C Working Draft. URL: https://www.w3.org/TR/rdf12-xml/

[SPARQL12-CONCEPTS]

SPARQL 1.2 Concepts. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-concepts/spec/

[SPARQL12-ENTAILMENT]

SPARQL 1.2 Entailment Regimes. Peter Patel-Schneider. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-entailment/

[SPARQL12-FEDERATED-QUERY]

SPARQL 1.2 Federated Query. Ruben Taelman; Gregory Williams. W3C. 26 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-federated-query/

[SPARQL12-GRAPH-STORE-PROTOCOL]

SPARQL 1.2 Graph Store Protocol. Andy Seaborne; Thomas Pellissier Tanon. W3C. 19 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-graph-store-protocol/

[SPARQL12-NEW]

What's New in SPARQL 1.2. The W3C RDF & SPARQL Working Group. W3C. W3C Editor's Draft. URL: https://w3c.github.io/sparql-new/spec/

[SPARQL12-PROTOCOL]

SPARQL 1.2 Protocol. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-protocol/

[SPARQL12-QUERY]

SPARQL 1.2 Query Language. Olaf Hartig; Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 29 January 2026. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-query/

[SPARQL12-RESULTS-CSV-TSV]

SPARQL 1.2 Query Results CSV and TSV Formats. Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-csv-tsv/

[SPARQL12-RESULTS-JSON]

SPARQL 1.2 Query Results JSON Format. Andy Seaborne; Ruben Taelman; Gregory Williams; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-json/

[SPARQL12-RESULTS-XML]

SPARQL 1.2 Query Results XML Format. Ruben Taelman; Dominik Tomaszuk; Thomas Pellissier Tanon. W3C. 27 December 2024. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-results-xml/

[SPARQL12-SERVICE-DESCRIPTION]

SPARQL 1.2 Service Description. Ruben Taelman; Gregory Williams. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-service-description/

[SPARQL12-UPDATE]

SPARQL 1.2 Update. Ruben Taelman; Andy Seaborne; Thomas Pellissier Tanon. W3C. 14 August 2025. W3C Working Draft. URL: https://www.w3.org/TR/sparql12-update/

↥